



**MANIPAL**  
**UNIVERSITY JAIPUR**  
*University under Section 2(f) of the UGC Act*

|                    |                           |
|--------------------|---------------------------|
| <b>NAME</b>        | <b>Hirdyansh Varshney</b> |
| <b>SEMESTER</b>    | <b>1st</b>                |
| <b>ROLL NUMBER</b> | <b>2414500172</b>         |
| <b>SESSSION</b>    | <b>September</b>          |
| <b>COURSE CODE</b> | <b>DCA1107</b>            |
| <b>COURSE NAME</b> | <b>C Programming</b>      |

### Set-I

#### Answer 1 :

Format specifiers in C play a crucial role in the printf function by defining how values are formatted and displayed. They act as placeholders within the output string, specifying the type of data being printed, such as integers, floating-point numbers, or characters. Each specifier begins with a % symbol followed by a character indicating the data type.

#### Key Format Specifiers:

##### 1. %d or %i:

- Represents signed integers.
- Example:
- `int num = 10;`

`printf("The number is %d", num);` // Output: The number is 10

##### 2. %f:

- Represents floating-point numbers.
- Example:
- `float pi = 3.14;`

`printf("Value of pi: %f", pi);` // Output: Value of pi: 3.140000

##### 3. %c:

- Represents a single character.

- Example:
- `char ch = 'A';`

`printf("Character: %c", ch); // Output: Character: A`

#### 4. **%s:**

- Represents a string.
- Example:
- `char str[] = "Hello";`

`printf("String: %s", str); // Output: String: Hello`

#### 5. **%x or %X:**

- Represents hexadecimal integers (lowercase for %x and uppercase for %X).
- Example:
- `int num = 255;`

`printf("Hexadecimal: %x", num); // Output: Hexadecimal: ff`

#### 6. **%o:**

- Represents octal integers.
- Example:
- `int num = 8;`

`printf("Octal: %o", num); // Output: Octal: 10`

#### 7. **%%:**

- Prints a literal percent sign.
- Example:

`printf("100%% Complete"); // Output: 100% Complete`

### **Width and Precision Modifiers:**

- Specifiers can be modified for width and precision. For example:

`printf("%8.2f", 3.14159); // Output: 3.14 (8 spaces wide, 2 decimals)`

### **Importance of Format Specifiers:**

- Ensure correct data type representation.
- Facilitate formatted output for better readability.
- Avoid runtime errors by maintaining type safety.

In conclusion, format specifiers are indispensable in C programming, allowing precise and structured data representation through the `printf` function.

## Answer 2 :

Decision control statements in C enable the program to make choices based on conditions. These statements determine the flow of execution by evaluating conditions that are either true or false.

### Types of Decision Control Statements:

#### 1. if Statement:

- Executes a block of code if a condition is true.
- Syntax:
  - if (condition) {
  - // Code to execute if condition is true
  - }
- Example:
  - int a = 10;
  - if (a > 5) {
  - printf("a is greater than 5");
  - }

#### 2. if-else Statement:

- Executes one block of code if the condition is true and another if false.
- Syntax:
  - if (condition) {
  - // Code if condition is true
  - } else {
  - // Code if condition is false
  - }
- Example:
  - int a = 10;
  - if (a > 15) {
  - printf("a is greater than 15");
  - } else {
  - printf("a is not greater than 15");
  - }

### 3. **if-else-if Ladder:**

- Used to check multiple conditions sequentially.
- Syntax:
- `if (condition1) {`
- `// Code for condition1`
- `} else if (condition2) {`
- `// Code for condition2`
- `} else {`
- `// Code if all conditions are false`
- `}`

- Example:
- `int marks = 85;`
- `if (marks >= 90) {`
- `printf("Grade: A");`
- `} else if (marks >= 75) {`
- `printf("Grade: B");`
- `} else {`
- `printf("Grade: C");`
- `}`

### 4. **switch Statement:**

- Tests a variable against multiple cases and executes the matching case.
- Syntax:
- `switch (expression) {`
- `case value1:`
- `// Code for value1`
- `break;`
- `case value2:`
- `// Code for value2`
- `break;`
- `default:`

```

        ○      // Code if no cases match
    }

    ○  Example:
    ○  int day = 3;
    ○  switch (day) {
    ○      case 1:
    ○          printf("Monday");
    ○          break;
    ○      case 2:
    ○          printf("Tuesday");
    ○          break;
    ○      case 3:
    ○          printf("Wednesday");
    ○          break;
    ○      default:
    ○          printf("Invalid day");
    }

```

### **Importance of Decision Control Statements:**

- Allow dynamic behavior based on input or conditions.
- Improve code efficiency and readability by controlling program flow effectively.

### **Answer 3 :**

An array in C is a collection of variables of the same data type stored in contiguous memory locations. Arrays simplify the management of large amounts of data by using a single name and an index to access each element.

### **Declaration of Arrays:**

- Syntax:

```
data_type array_name[size];
```

- Example:

```
int numbers[5];
```

### **Initialization of Arrays:**

1. **During Declaration:**

- Example:

```
int numbers[5] = {1, 2, 3, 4, 5};
```

## 2. Partial Initialization:

- Example:

```
int numbers[5] = {1, 2}; // Remaining elements initialized to 0
```

## 3. Dynamic Initialization:

- Example:
- `int numbers[5];`
- `for (int i = 0; i < 5; i++) {`
- `numbers[i] = i * 10;`

```
}
```

## Accessing Array Elements:

- Use indices starting from 0.
- Example:

```
printf("First element: %d", numbers[0]);
```

## Multidimensional Arrays:

- Syntax:

```
data_type array_name[rows][columns];
```

- Example:

```
int matrix[2][2] = {{1, 2}, {3, 4}};
```

## Importance of Arrays:

- Facilitate efficient storage and management of data.
- Enable bulk data operations, like sorting and searching.
- Simplify code by avoiding the need for multiple variables for similar data.

In conclusion, arrays are a fundamental data structure in C programming, offering a systematic way to handle large datasets and perform operations on them efficiently.

## Set-II

### Answer 4 :

Null-terminated strings in C are a type of character array terminated with a null character (`\0`). This null character signals the end of the string, differentiating it from other character arrays that might not have this termination.

## Characteristics of Null-Terminated Strings:

### 1. Termination:

- Always end with a `\0` character, which marks the end of the string.
- Example:

```
char str[] = "Hello"; // Equivalent to {'H', 'e', 'l', 'l', 'o', '\0'}
```

### 2. Accessing Elements:

- Elements are accessed using indices, starting from 0.
- Example:

```
printf("%c", str[0]); // Output: H
```

### 3. String Manipulation Functions:

- C provides several functions in `string.h` for handling null-terminated strings:
  - `strlen()`: Finds string length.
  - `strcpy()`: Copies one string to another.
  - `strcmp()`: Compares two strings.

## Difference from Regular Character Arrays:

- A regular character array may or may not have a null terminator.
- Example:

```
char arr[5] = {'H', 'i', '!', ' ', '?'}; // Not a null-terminated string
```

- Operations like `printf()` or `strlen()` behave unpredictably without a `\0`.

### Example:

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char str1[] = "Hello";
```

```
    char str2[] = {'H', 'e', 'l', 'l', 'o'}; // Not null-terminated
```

```
    printf("String 1: %s\n", str1); // Output: Hello
```

```
    printf("Length of str1: %lu\n", strlen(str1)); // Output: 5
```

```

// Using str2 might cause unpredictable behavior
printf("String 2: %s\n", str2); // May print garbage after "Hello"

return 0;
}

```

Null-terminated strings are a core concept in C programming, essential for text manipulation and input/output operations.

### Answer 5 :

Recursion in C is a programming technique where a function calls itself directly or indirectly to solve a problem. It is widely used for problems that can be broken down into smaller, similar sub-problems.

### Characteristics of Recursion:

#### 1. Base Case:

- A condition to stop the recursion, preventing infinite loops.
- Example:
- if (n == 0) {
- return 1;
- }

#### 2. Recursive Case:

- The part of the function where recursion is applied to reduce the problem size.

### Steps in Recursive Execution:

1. Function calls itself with a reduced problem.
2. Base case terminates recursion.
3. Each call's return value is used to compute the final result.

### Example: Factorial Calculation

```
#include <stdio.h>
```

```

int factorial(int n) {
    if (n == 0) {
        return 1; // Base case
    } else {

```



```

        return n * factorial(n - 1); // Recursive case
    }
}

int main() {
    int num = 5;
    printf("Factorial of %d is %d", num, factorial(num));
    return 0;
}

```

**Explanation:**

1. factorial(5) calls factorial(4), and so on until factorial(0).
2. The base case returns 1, and recursive calls compute the result.
3. Final result:  $5 * 4 * 3 * 2 * 1 = 120$ .

**Key Points:**

- Recursion simplifies complex problems like tree traversal, sorting, etc.
- Proper base cases are essential to avoid infinite loops.
- Overuse can lead to stack overflow errors.

In conclusion, recursion is a powerful concept in C programming, enabling elegant solutions for problems that exhibit self-similar structures. However, it should be used judiciously to maintain efficiency.

**Answer 6 :**

Selection sort is a simple sorting algorithm that repeatedly selects the smallest (or largest) element from the unsorted portion of the array and moves it to the sorted portion. It works by dividing the array into two parts: a sorted section and an unsorted section.

**Algorithm:**

1. Start with the first element as the minimum.
2. Compare the minimum with the remaining elements to find the smallest value.
3. Swap the smallest value with the first element of the unsorted section.
4. Move the boundary of the sorted section one element to the right.
5. Repeat until the array is sorted.

**Example:**

Consider the array [64, 25, 12, 22, 11]:

1. Initially, 11 (smallest element) is swapped with 64.
2. Next, 12 is swapped with 25.
3. The process continues until the array becomes [11, 12, 22, 25, 64].

**Code Implementation:**

```
#include <stdio.h>
```

```
void selectionSort(int arr[], int n) {  
    int i, j, minIndex, temp;  
  
    for (i = 0; i < n - 1; i++) {  
        minIndex = i; // Assume the first element is the minimum  
  
        // Find the minimum element in the remaining unsorted array  
        for (j = i + 1; j < n; j++) {  
            if (arr[j] < arr[minIndex]) {  
                minIndex = j;  
            }  
        }  
  
        // Swap the found minimum element with the first element  
        temp = arr[minIndex];  
        arr[minIndex] = arr[i];  
        arr[i] = temp;  
    }  
}  
  
void printArray(int arr[], int size) {  
    for (int i = 0; i < size; i++) {  
        printf("%d ", arr[i]);  
    }  
}
```

```
        printf("\n");
    }

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: \n");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

### **Output:**

Original array:

64 25 12 22 11

Sorted array:

11 12 22 25 64

### **Complexity:**

- **Time Complexity:**  $O(n^2)$  for worst and average cases.
- **Space Complexity:**  $O(1)$ , as it is an in-place sorting algorithm.

### **Applications:**

- Suitable for small datasets.
- Useful in scenarios where memory is limited due to its in-place nature.

In conclusion, selection sort is an intuitive and straightforward sorting algorithm, but its inefficiency on larger datasets limits its practical applications.